

Java Full Stack Developer Interview Guide

Targeted Technical Q&A based on Java, Spring Boot, Spring Security, and React Experience

1. Core Java & OOP Concepts

Q1: How do you handle exception handling in a layered Spring Boot architecture?

Answer: I use a centralized exception handling mechanism with `@ControllerAdvice` and `@ExceptionHandler`. Custom exceptions (e.g., `ResourceNotFoundException`) are thrown from service layers and caught globally to return standard HTTP error responses (JSON payload containing timestamps, status codes, and error descriptions).

Q2: Explain how you utilize Java Collections and Multithreading in backend apps.

Answer: I frequently use `HashMap` and `ConcurrentHashMap` for caching or in-memory lookups. For asynchronous task processing or batch operations (such as notifications or reports), I leverage Java Multithreading via `ExecutorService` or Spring's `@Async` annotation with proper thread pool configurations.

2. Spring Boot, JPA, & Hibernate

Q3: How do you optimize database performance when dealing with JPA/Hibernate N+1 query problems?

Answer: The N+1 select problem occurs when fetching relational entities (e.g., Show and Movie). I solve this using JPQL/Criteria queries with `JOIN FETCH`, or by specifying `@EntityGraph` annotations in Spring Data JPA repositories to load relationships eagerly in a single query.

Q4: Describe how you structured the CineBook movie ticket booking project backend.

Answer: CineBook follows a strict layered architecture: **Controller** (handles HTTP requests and routing), **Service** (encapsulates business logic like seat availability checks and pricing rules), and **Repository** (handles database communication using Spring Data JPA and Hibernate with PostgreSQL).

3. Spring Security & JWT Authentication

Q5: How does JWT-based authentication work in your Spring Boot applications?

Answer: Upon successful login, authentication tokens are generated using a secret key and signed via JJWT library. Subsequent requests send the token in the `Authorization: Bearer <token>` header. A custom `OncePerRequestFilter` intercepts requests, validates the JWT, extracts user roles, and populates the `SecurityContextHolder`.

Q6: How do you ensure password security in production?

Answer: Passwords are never stored in plain text. I use `BCryptPasswordEncoder` from Spring Security to hash passwords with a secure salt before persisting them into MySQL/PostgreSQL databases.

4. Frontend Integration & Real-World Projects

Q7: How did you handle payment gateway integration (Razorpay) in Orisnz-Fleet / CineBook?

Answer: The backend initiates a payment order via Razorpay REST APIs and returns an order ID. The React frontend triggers the Razorpay checkout modal. Upon successful client transaction, webhook or response verification signatures are validated securely on the backend before confirming bookings.

Q8: How do you connect a React/Vite frontend with a Spring Boot REST backend?

Answer: I configure Axios instances with base URLs and request interceptors to automatically attach JWT tokens from local storage. CORS policies are explicitly configured in Spring Boot using `CorsRegistry` to allow secure cross-origin communication between the Vite dev/prod server and the backend API.